

Kubevirt 调研

背景

当前针对 **vm** 和容器存在两套调度平台，**vm** 由 **openstack** 调度，容器由 **k8s** 调度，存在资源浪费。同时随着业务的全面上云，大部分用户业务都开始容器化，所以未来 **k8s+容器** 肯定是业务发布的主流选择，业界也基本成型。

虽然 **vm** 的使用场景会被压缩，但是 **vm** 作为一个常用的运行时，未来也会长期存在较长时间。

简介

Kubevirt 是 Redhat 开源一套以容器方式运行虚拟机的项目，通过 **kubernetes** 云原生来管理虚拟机生命周期。

Kubevirt 是通过使用自定义资源（CRD）和其它 **Kubernetes** 功能来无缝扩展现有的集群，以提供一组可用于管理虚拟机的虚拟化的 API。

Kubevirt 提供如下几种 CRD 资源：

对象名称	功能
<u>VirtualMachineInstance</u>	虚拟机管理的最小资源，一个 <u>VMI</u> 对象即表示一台正在运行的 <u>VM</u>
<u>VirtualMachine</u>	为群集内的 <u>VMI</u> 提供生命周期管理功能，确保虚拟机实例的状态，并且它与 <u>VMI</u> 1: 1
<u>VirtualMachineInstanceMigration</u>	主要提供热迁移功能

VMI 资源样例：

```
---
apiVersion: kubevirt.io/v1
kind: VirtualMachineInstance
metadata:
  labels:
    special: vmi-fedora
    name: vmi-fedora
spec:
  domain:
  devices:
```

```

disks:
- disk:
  bus: virtio
  name: containerdisk
- disk:
  bus: virtio
  name: cloudinitdisk
rng: {}
machine:
  type: ""
resources:
  requests:
    memory: 1024M
terminationGracePeriodSeconds: 0
volumes:
- containerDisk:
  image: registry:5000/kubevirt/fedora-cloud-container-disk-demo:devel
  name: containerdisk
- cloudInitNoCloud:
  userData: |-
    #cloud-config
    password: fedora
    chpasswd: { expire: False }
  name: cloudinitdisk

```

说明：

1.kubevirt 是不是一个 vm 管理平台的替代品，和 OpenStack 还有 ovirt 等虚拟化管理平台的区别。

简单来说：kubevirt 只是用 k8s 管 vm，其中会复用 k8s 的 cni 和 csi，所以只是用 operator 的方式来操作 vm，他不去管网络和存储等。所以和 OpenStack 中包含 nova，neutron，cinder 等不一样，可以理解成 kubevirt 是一个 k8s 框架下的，用 go 写的 nova vm 管理组件。

2.kubevirt 和 kata 的区别。

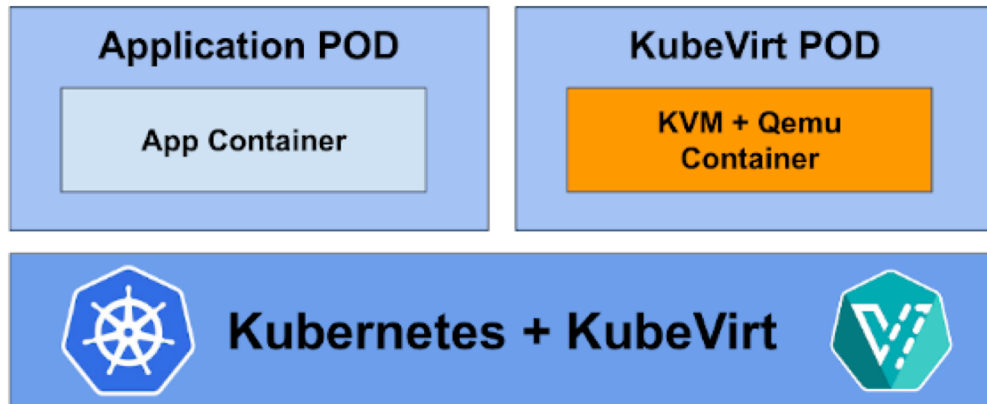
简单来说：kata 是有着 vm 的安全性和隔离性，以容器的方式运行，有着容器的速度和特点，但不是一个真正的 vm，而 kubevirt 是借用 k8s 的扩展性来管 vm，你用到的是一个真正的 vm。

3.为啥要用 k8s 管 vm，而不是用 OpenStack 管容器。

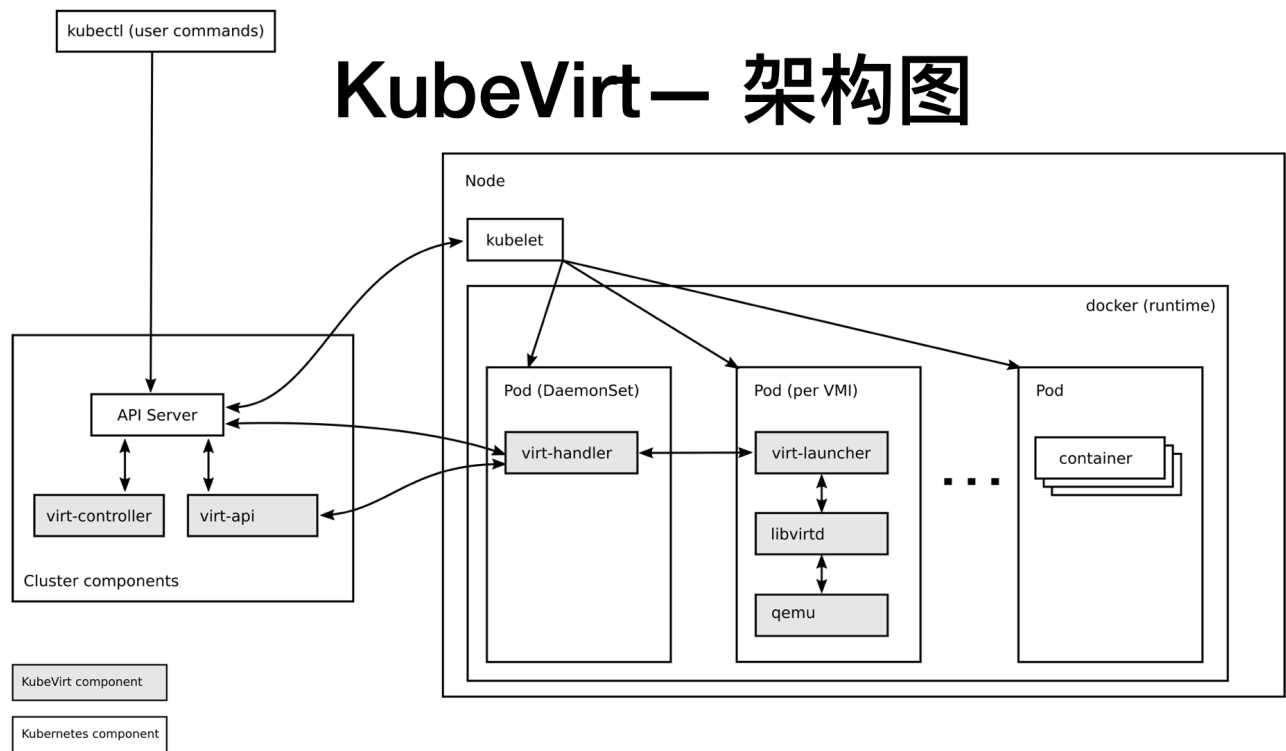
简单来说：k8s+容器是未来的主流方向，但是由于历史和业务需要，vm 也会存在很长时间，所以我们一套支持 vm 的容器管理平台 k8s，而不是需要一套支持容器的 vm 管理平台例如 OpenStack 管容器 Magnum 这种类似项目。

架构

虚拟化—KubeVirt



- 主要 CRD 对象/资源: VirtualMachine、VirtualMachineInstance、DataVolume、ContainerDisk



从 kubevirt 架构看如何创建虚拟机

Kubevirt 架构如上图所示，由4部分组件组成。从架构图看出 kubevirt 创建虚拟机的核心就是创建了一个特殊的 pod virt-launcher，其中的子进程包括 libvirt 和 qemu。做过 openstack nova 项目的应该比较习惯于一台宿主机中运行一个 libvirtd 后台进程，kubevirt 中采用每个 pod 中一个 libvirt 进程是去中心化的模式避免因为 libvirtd 服务异常导致所有的虚拟机无法管理。

功能

Kubevirt 组件介绍

与 OpenStack 类似，kubevirt 每个组件负责不同的功能，不同点是资源调度策略由 k8s 去管理，其中主要组件如下：virt-api、virt-controller、virt-handler、virt-launcher。

名称	功能
virt-api	(1) HTTP API Server 作为所有涉及虚拟化相关的处理流程的入口 (Entry Point)，负责更新、验证 VMI CRDs； (2) 提供 RESTful API 来管理集群中虚拟机，Kubevirt 采用 CRD 的工作方式，virt-api 提供自定义的 API 请求处理流程，如 VNC、Console、Start/Stop 虚拟机；

virt-controller	(1) 该控制器负责监控虚拟机实例 VMI 对象和管理集群中每个虚拟机实例 VMI 的状态以及与其相关的 Pod ； (2) VMI 对象将在其生命周期内始终与容器关联，但是，由于 VMI 的迁移，容器实例可能会随时间变化 ；
virt-handler	(1) 在 K8S 的计算节点上，virt-handler 运行于 Pod 中，作为 DaemonSet ； (2) 类似于 virt-controller 都是响应式的，virt-handler 负责监控每个虚拟机实例的状态变化，一旦检测到状态变化就响应并确保相应操作能达到所需（理想）状态 ； (3) virt-handler 负责以下几方面：保持集群级 VMI Spec 与相应 libvirt 域之间的同步；报告 Libvirt 域状态和集群 Spec 的变化；调用以节点为中心的插件以满足 VMI Spec 定义的网络和存储要求 ；
Virt-launcher	(1) 每个虚拟机实例（VMI）对象都会对应一个 Pod，该 Pod 中的基础容器中运行着 Kubevirt 核心组件 virt-launcher ； (2) K8S 或者 Kubelet 是不负责运行 VMI 的运行的，取而代之的是，群集中每个节点上的守护进程会负责为每个 Pod 启动一个与 VMI 对象关联的 VMI 进程，无论何时在主机上对其进行调度。 (3) virt-launcher Pod 的主要功能是提供 cgroups 和名称空间并用于托管 VMI 进程。 (4) virt-handler 通过将 VMI 的 CRD 对象传递给 virt-launcher 来通知 virt-launcher 启动 VMI。然后 virt-launcher 在其容器中使用本地 libvirtd 实例来启动 VMI。从此开始，virt-

	launcher 将监控 VMI 进程，并在 VMI 实例退出后终止； (5) 如果 K8S 的 Runtime 在 VMI 退出之前试图关闭 virt-launcher Pod 时，virt-launcher 会将信号从 K8S 转发给 VMI 进程，并尝试推迟 pod 的终止，直到 VMI 成功关闭。
Libvirtd	(1) 每个 VMI 实例对应的 Pod 都会有一个 libvirtd 实例； (2) virt-launcher 借助于 libvirtd 来管理 VMI 实例的生命周期；

虚拟机创建流程（重要）

1. client 发送创建 VMI 命令达到 k8s API server.
2. K8S API 创建 VMI
3. virt-controller 监听到 VMI 创建时，根据 VMI spec 生成 pod spec 文件，创建 pods
4. k8s 调度创建 pods
5. virt-controller 监听到 pods 创建后，根据 pods 的调度 node，更新 VMI 的 nodeName
6. virt-handler 监听到 VMI nodeName 与自身节点匹配后，与 pod 内的 virt-launcher 通信，virt-launcher 创建虚拟机，并负责虚拟机生命周期管理

存储

kubevirt 提供很多种存储方式，存储就决定了你使用的虚拟机镜像。

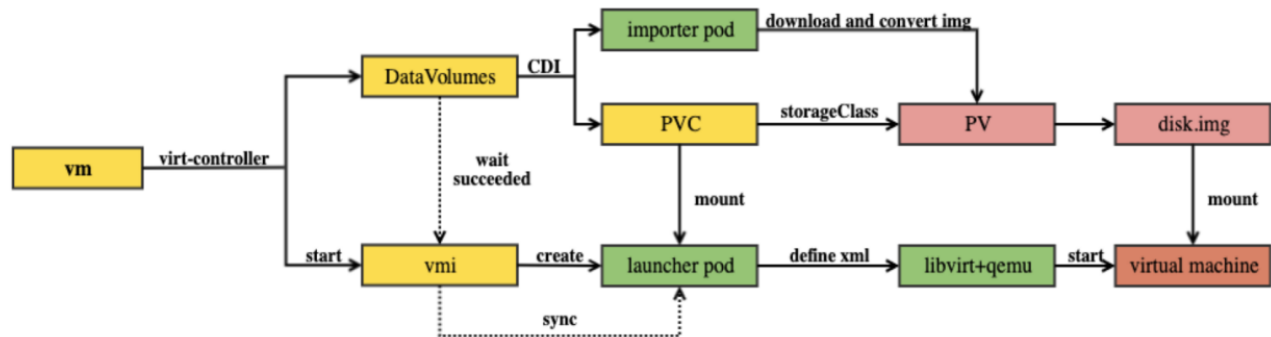
kubevirt 主要支持的几种存储：

- 1 cloudinit: 注入文件数据，可以用于执行初始化 user 脚本，在 VMI 内部体现为第二块盘（相当于 openstack nova 组件的密钥注入功能）
- 2 configmap/secret 与 kubernetes 保持一致
- 3 emptyDisk: 类似于 kubernetes 的 emptyDir
- 4 hostDisk: 类似于 hostPath，把 host 上面的一个镜像文件或者一块存储空间分配给 VMI 使用
- 5 containerDisk: 把虚拟机镜像保存为容器镜像，属于非持久化存储，数据不可变，适合不需要保存系统盘数据的场景

6 **dataVolume**: 持久化存储, 具备镜像自动导入能力。依赖 CDI (Containerized Data Importer) 实现此功能, 能够自动创建带有系统镜像数据的 **pv**, 镜像格式只支持 **raw** 格式, 非 **raw** 格式的镜像 (比如 **qcow2**, **iso**) 在导入过程中会自动转换为 **raw** 格式, 类似于 **openstack** 的 **cinder** 和 **nova-compute** 从 **glance** 拉取镜像且将镜像制作为系统盘的方式

7 **PVC**: **PV** 类型可支持 **block** 和 **filesystem**。为 **filesystem** 时, 将使用 **PVC** 上的 **disk.img**, 格式为 **RAW** 格式的文件作为硬盘挂载到 **pod** 使用 (相当于 **hostDisk** 使用方式)。**block** 模式时, 使用 **block volume** 直接作为原始块设备提供给虚拟机。也支持对接 **CSI**。

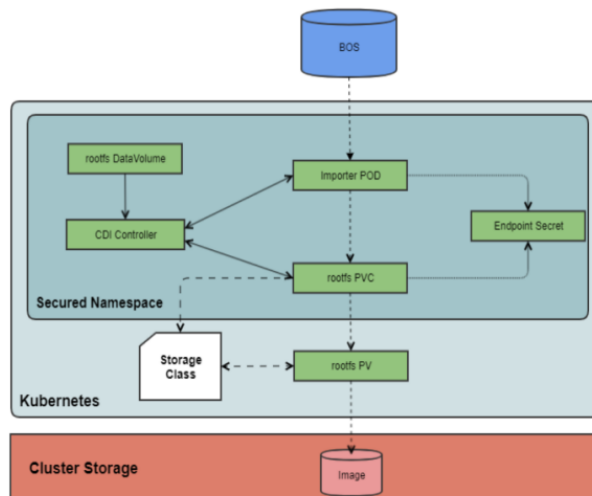
重点关注的是 **dataVolume** 的持久化存储:



流程如下:

1. 创建一个 **volumeBindingMode=WaitForFirstConsumer** 的 **StorageClass**
2. 用户创建有 **DataVolumeTemplate** 模板的虚拟机
3. `Kubevirt` 创建 **DataVolume**
4. `CDI` 发现新的 DV 有未绑定的 标志为 **volumeBindingMode=WaitForFirstConsumer** 的 **PVC**, 把 DV 的标志设置为 `WaitForFirstConsumer` 等待外部操作来绑定 **PVC**.
5. `Kubevirt` 查看有 `WaitForFirstConsumer` 标志的 DV, 创建一个临时 **pod** (也就是一个没有 **VM** 载荷的且有 `kubevirt.io/ephemeral-provisioning` annotation 的 **virtlauncher pod**) 用于创建相应的 **PV**
6. **Kubernetes** 调度临时 **Pod** (选择的节点满足所有 **VM** 要求), **Pod** 需要与 **VM** 相同的 **PVC**, 因此 **kubeneretes** 必须在正确启动 **Pod** 之前在正确的节点上设置 **PV** 并将其绑定到 **PVC**
7. `CDI` 发现 **PVC** 已绑定, 将 **DV** 状态更改为 "ImportScheduled" (或克隆/上载), 并尝试启动辅助容器
8. `Kubevirt` 看到 **DV** 状态为 `ImportScheduled`, 它可以终止临时 **importer pod**
8. `CDI` 执行导入, 将 **DV** 标记为 `Succeeded`
9. `Kubevirt` 创建 **virtlauncher pod** 来启动虚拟机

```
dataVolumeTemplates:
- metadata:
  name: alpine-dv
  spec:
    pvc:
      accessModes:
      - ReadWriteOnce
      resources:
        requests:
          storage: 2Gi
      storageClassName: local
    source:
      http:
        url: http://cdi-http-import
```



如上模板所示，dataVolumeTemplates 的 source 可以是 http 或者 pvc，也就是说可以用 http 协议从某个地址拉取镜像，或者是从某个 pvc 中拉取镜像。

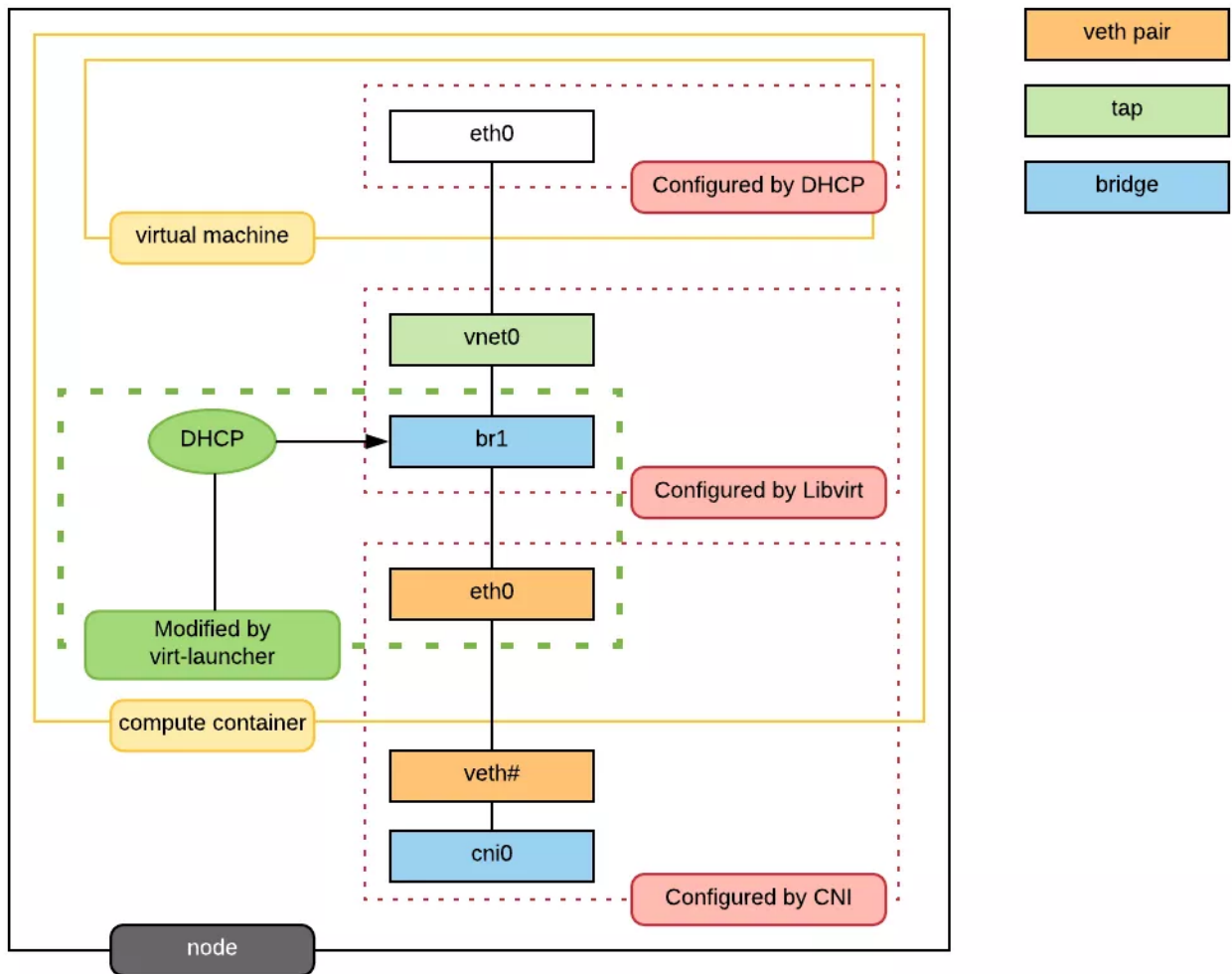
存储优缺点：

- 1 cloudinit: 创建的设备比较小，一般只有几百 k
- 2 hostDisk:VMI 使用 hostDisk 不支持迁移
- 3 dataVolume: 能够自动化创建系统盘 PV；但是如果镜像格式不是 raw 或者镜像比较大，创建起来比较慢。

与 openstack 的功能对比：

- 1 挂载：支持热插拔挂载数据盘
- 2 卸载：支持热插拔卸载数据盘
- 3 迁移：当所有卷均已共享且 VMI 没有本地磁盘时，才可以迁移。openstack 是可以支持本地镜像起的虚拟机的迁移
- 4 快照：目前有一项 alpha 阶段的功能，VolumeSnapshot 和 VirtualMachineRestore，此功能基于 CSI 的能力来进行 VMI 资源的备份和恢复
- 5 克隆：目前支持一种叫 smart-clone 方式导入，可以加快镜像自动导入的速度，但是依赖于存储。

网络



kubernetes 是 Kubevirt 底座，提供了管理容器和虚拟机的混合部署的方式，存储和网络也是通过集成到 kubernetes 中，VMI 使用了 POD 进行通信。为了实现该目标，KubeVirt 的对网络做了特殊实现。虚拟机具体的网络如图所示，virt-launcher Pod 网络的网卡不再挂有 Pod IP，而是作为虚拟机的虚拟网卡的与外部网络通信的交接物理网卡。

在当前的场景我们使用经典的大二层网络模型，用户在一个地址空间下，VM 使用固定 IP，在 OpenStack 社区，虚拟网络方案成熟，OVS 基本已经成为网络虚拟化的标准。所以我们选择目前灵雀云（alauda）开源的网络方案：Kube-OVN，它是基于 OVN 的 Kubernetes 网络组件，提供了大量目前 Kubernetes 不具备的网络功能，并在原有基础上进行增强。通过将 OpenStack 领域成熟的网络功能平移到 Kubernetes，来应对更加复杂的基础环境和应用合规性要求。

说明：

1. 主要支持两种模式：默认 pod 网络、多网卡 multus
2. 官方支持 ovs-cni；理论支持 kube-ovn（待验证）
3. 完全复用 Kubernetes 的 DNS、Service 机制

监控

Kube-handler 会去调用当前节点下所有虚拟机的 libvirt API，获取虚拟机的监控指标，并提供 metrics 接口，最后通过 kubevirt-prometheus-metrics 聚合所有节点的 kube-handler 的指标数据，提供给 prometheus 使用。

迁移

通过 CRD: VirtualMachineInstanceMigration (VMIM) 来实现动态迁移

```
apiVersion: kubevirt.io/v1alpha3
kind: VirtualMachineInstanceMigration
metadata:
  name: migration-job
spec:
  vmiName: vmi-fedora
```

这种迁移形式其实并没有指定迁移到哪些节点上，内部逻辑应该只是重新调度 virt-lanucher pod，其中 kubeirt-config 可以对此进行迁移的相关限制，比如迁移的频率（同时只能几个节点进行迁移），迁移的网络速率（因为可能实际到数据磁盘复制），通过 VMIM 也能查看到迁移进度

对比

Kubevirt VS Kata Container

当然 Kube-virt 并不是目前唯一的可以在 Kuberentes 上实践虚拟化的技术。我们把它也和目前比较流行的 kata container 做了一个对比，方便用户根据实际情况来做选择。

	kubevirt	Kata-container
支持虚拟机系统	Linux, windows	linux
安全程度	虚拟机安全	虚拟机安全
启动速度	虚拟机启动速度	接近容器启动速度慢
虚拟机完整功能	支持	不支持
支持K8s对象 (deploy, sts)	有类似CRD (virtualMachineReplicase)	是
支持常规健康检查	是	是

总结

同时我们在调研过程遇到一些问题，比如重启数据丢失、VMI 重启和热迁移后 IP 改变、镜像导入数据缓慢、VMI 启动调度缓慢、热迁移网络与存储支持等等。Kubevirt 通过 CRD 方式将 VM 管理接口接入到 k8s 集群，而 POD 使用 libvirt 管理 VMI，如容器一样去管理 VMI，最后通过标准化插件方式管理调度网络和存储资源对象，将其整合在一起形成一套 具有 K8s 管理虚拟化的技术栈。

补充说明

1. pod 中资源如何分配

内存资源

计算 pod 需要的内存资源，包括客户机所需的内存以及 Qemu 和 OS 开销所需的内存。

- (1) 添加页表所需的内存，即 memory request 所需的内存
- (2) 添加动态库所需内存，固定138M
- (3) 添加 vCPU 所需内存，即 vcpu request 所需的内存，每个 vcpu 分配8M
- (4) 添加 IO 线程所需内存，8M
- (5) 添加视频 RAM 所需内存，16M
- (6) VFIO 设备所需内存，1G。VFIO 除了 MMIO 内存空间外，还需要锁定所有 guest RAM 以允许 DMA。1G 通常是 x86系统上保留的 MMIO 空间的大小。

CPU 资源

计算 pod 需要的 cpu 资源。

It counts sockets * cores * threads

2. 特性支持

支持 gpu，支持 sriov

3. 虚拟机镜像

方式一：ContainerDisk

```
spec:
  domain:
    devices:
      disks:
        - disk:
            bus: virtio
```

```

    name: containerdisk
    rng: {}
    machine:
      type: ""
    resources:
      requests:
        memory: 1024M
    terminationGracePeriodSeconds: 0
    volumes:
    - containerDisk:
        image: registry:5000/kubevirt/fedora-cloud-container-disk-demo:devel
        name: containerdisk

```

先把本地镜像制作成容器镜像，virt-controller 会在 pod 定义中创建 ContainerDisk 的 container，container 中的 entry 服务负责将 `spec.volumes.containerDisk.image` 转化为 qcow2 格式，路径为 pod 根目录。virt-launcher 创建虚机时会使用这个虚机镜像。

目前看到的例子都是基于本地 host 存在镜像的场景，故尚不确认是否具备镜像统一管理的能力

方式二：dataVolume

```

dataVolumeTemplates:
- metadata:
    creationTimestamp: null
    name: alpine-dv
  spec:
    pvc:
      accessModes:
      - ReadWriteOnce
      resources:
        requests:
          storage: 2Gi
      storageClassName: local
    source:
      http:
        url: http://cdi-http-import-server.kubevirt/images/alpine.iso

```

dataVolume 是 kubevirt 下的一个子项目 containerized-data-importer(CDI)，dataVolume 具备镜像自动导入能力，可以定义 `source` 即 image/data 来源可以是 `http` 或者 `s3` 的 URL，CDI controller 会自动将 image 转化并拷贝到 PVC 文件系统 `/disk/`，类似于 openstack 的 cinder 和 nova-compute 从 glance 拉取镜像且将镜像制作为系统盘的方式。

refs:

<https://zhuanlan.zhihu.com/p/107789413>

<https://mp.weixin.qq.com/s/RGYWwzi9Q9fBiGOqf19jlg>

<https://remimin.github.io/2018/09/14/kubevirt/>

<https://blog.csdn.net/oliverlyn/article/details/89765774>

<https://kubevirt.io/user-guide/welcome/>